



Zagazig National University

Faculty of Engineering

Department of Mechatronics Engineering

RFID Attendance System Using ESP8266 and Google Sheets

Prepared by:

- | | |
|--------------------------------------|--|
| 1) Youssef Ehab Taha El-Morsi | 5) Basmala Hsony Farouk Ibrahim |
| 2) Ali Waleed Ali Zied | 6) Malak Mohamed Abdelhakam |
| 3) Momen Ali Fekry | 7) Yasmin Jaser Mohamed |
| 4) Hadi Ibrahim Alhadi | |

Supervised by:

Dr. Ahmed Osman

December 2025

Detailed Report — RFID Attendance System Using ESP8266 and Google Sheets

Project Summary

This project is an electronic attendance system that uses an RFID reader (RC522) connected to an **ESP8266** (NodeMCU/WeMos board), a 16×2 I2C LCD display for status messages, a buzzer or LED for feedback, and Google Sheets for logging attendance records through a Google Apps Script (WebApp). All components are mounted on a wooden board (Ablakash) for stable installation on a desk or wall.

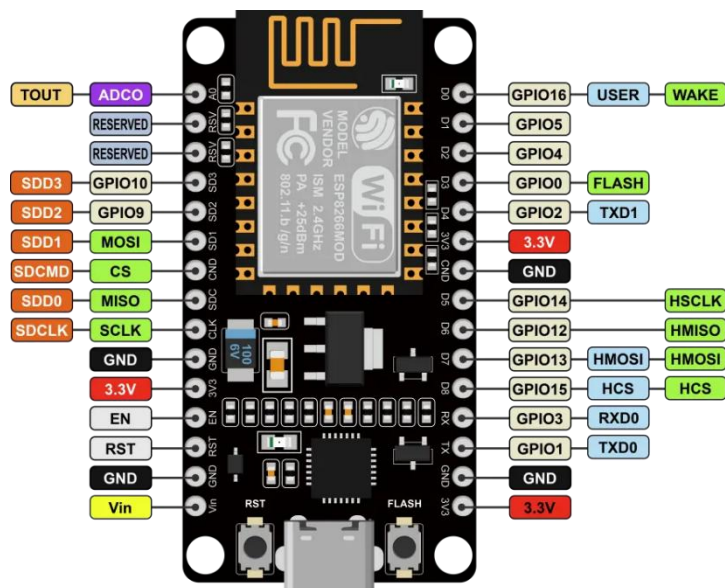
This document includes: a complete bill of materials, reasons for choosing each component, full wiring details (pinout + connection map), detailed explanation of all code files, how data is sent to Google Sheets, assembly instructions, testing steps, troubleshooting, and future improvement suggestions.

1) Bill of Materials (BOM)

- **ESP8266 NodeMCU** (or any compatible ESP8266 board) — 1 pc
 - **RFID Reader RC522 / MFRC522** — 1 pc
 - RFID Tags/Cards (MIFARE 1K) — at least 2
 - **16×2 I2C LCD Display** (I2C address usually 0x27) — 1 pc
 - Buzzer or LED indicator — 1 pc
 - Jumper wires — assorted
 - 5V USB power supply
 - Screws and plastic spacers for mounting components on the wooden board
 - Optional: Housing or plastic covers for clean installation
-

2) Why These Components Were Chosen

- **1-ESP8266 NodeMCU**: Easy to program using Arduino IDE, built-in Wi-Fi, perfect for sending data to Google Sheets using HTTPS.
- **2-RC522 (MFRC522)**: Inexpensive and widely supported RFID module, supports 2-reading/writing MIFARE 1K cards.
- **3-I2C LCD 16×2**: Displays user instructions and attendance confirmation; using I2C reduces wires (only SDA & SCL).
- **4-Buzzer / LED**: Provides user feedback on successful card scanning.
- **5-Google Sheets + Apps Script**: Provides free, cloud-based logging accessible from anywhere; no need for a backend server.



(1)



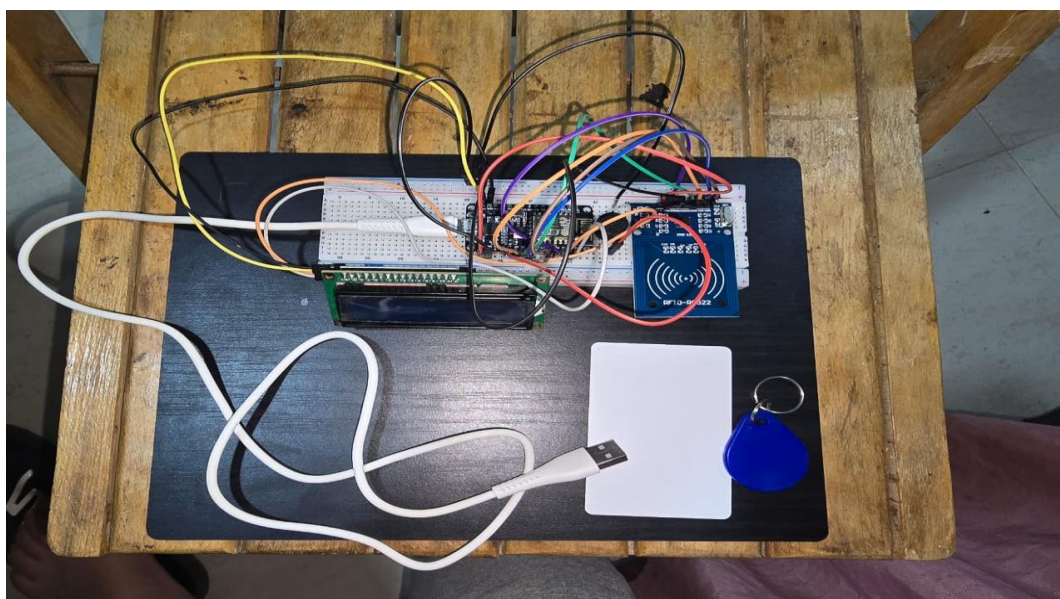
(2)



(3)

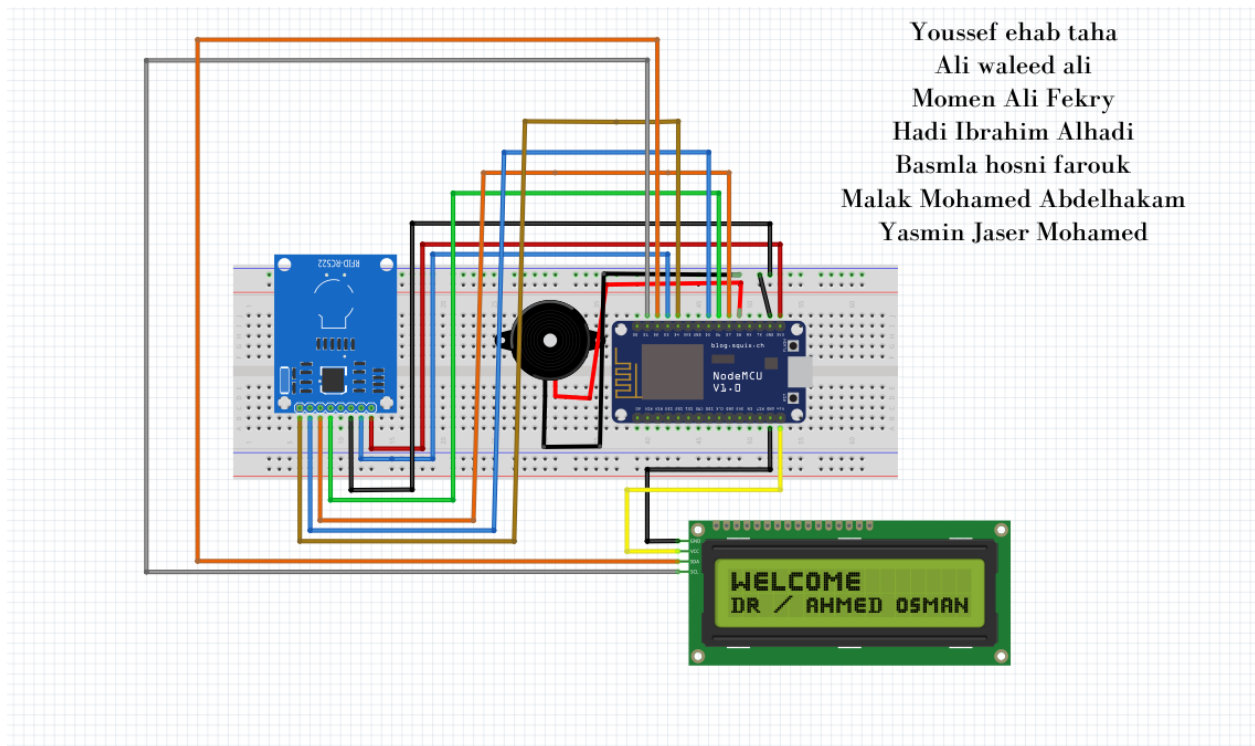


(4)



3) General Notes Before Wiring

1. NodeMCU/Wemos pin labels (D0–D8) are different from actual GPIO numbers — a mapping table is provided below.
2. RC522 operates at **3.3V only** — do NOT connect it to 5V.
3. SDA (SS) and RST pins must match the pin definitions used in your code.
4. I2C LCD modules often accept 5V, but ESP8266 logic levels are 3.3V — most modules handle this, but verify your unit.



4) NodeMCU Pinout (Dn → GPIO Mapping)

Board Label	GPIO No.	Function in Project
D0	GPIO16	Not used
D1	GPIO5	I2C SCL for LCD
D2	GPIO4	I2C SDA for LCD (and sometimes used for buzzer)
D3	GPIO0	RC522 RST
D4	GPIO2	RC522 SDA/SS
D5	GPIO14	SPI SCK
D6	GPIO12	SPI MISO
D7	GPIO13	SPI MOSI
D8	GPIO15	Optional LED indicator
3V3	3.3V	Power for ESP8266 & RC522
GND	GND	Common ground

5) Wiring Instructions (Text Description)

RC522 Wiring (SPI)

- RC522 **SDA/SS** → NodeMCU **D4** (GPIO2)
- RC522 **SCK** → **D5** (GPIO14)
- RC522 **MOSI** → **D7** (GPIO13)
- RC522 **MISO** → **D6** (GPIO12)
- RC522 **RST** → **D3** (GPIO0)
- RC522 **3.3V** → **3V3**
- RC522 **GND** → **GND**

16×2 I2C LCD Wiring

- LCD **SDA** → **D2** (GPIO4)
- LCD **SCL** → **D1** (GPIO5)
- LCD **VCC** → **3V3** or **5V** (depending on your module)
- LCD **GND** → **GND**

Buzzer / LED

- Buzzer + → **D2** (or pin defined as BUZZER in code)
- Buzzer – → **GND**
- If using LED: add a **220Ω resistor** in series.

Power

- NodeMCU → USB 5V power supply (minimum 500mA recommended).

6) Code Overview

6.1 Program_for_data_stored_in_RFID_Tag (RFID Write Program)

Purpose: Write student information (Roll number, Name, Department, Level, etc.) into specific memory blocks of the RFID card.

Key Points:

- Uses MIFARE 1K blocks: **4, 5, 6, 8, 9**.
- Each block stores 16 bytes.
- Requires authentication using default Key A `FF FF FF FF FF FF`.
- Inputs are read from Serial until a # character marks the end of a field.

Why This Code Is Needed: It prepares the cards with pre-stored student data that the attendance system will later read.

6.2 Program_for_Student_Attendance_System (RFID Read + Google Sheets Logging)

Purpose: Read card data, create a JSON payload, and send it via HTTPS POST to a Google Apps Script to log attendance.

Key Points:

- Reads the same blocks (4,5,6,8,9).
 - Prevents repeated scans using `lastUID` and `lastScanTime`.
 - Sends JSON payload: `{ "command": "insert_row", "sheet_name": "Sheet1", "values": "roll,name,..." }`
 - Uses `HTTPSRedirect` library to send secure requests.
-

6.3 Google Apps Script (WebApp)

Purpose: Receive POST requests from ESP8266 and insert a new row in Google Sheets.

Key Points:

- Uses `JSON.parse(e.postData.contents)`.
- Splits incoming values and inserts them in row 2.
- Includes timestamp and date.
- Must be deployed as **Web App**.

Security Note: Avoid setting the WebApp to "Anyone, even anonymous" unless necessary.

7) Required Edits Before Running the System

1. **ESP8266 Code:** Update `GScriptId` to your Google Apps Script deployment ID.
 2. **Google Apps Script:** Update `ss` to your spreadsheet ID.
 3. **Wi-Fi Settings:** Set correct `ssid` and `password`.
 4. Check LCD address (0x27 or 0x3F) and update if needed.
-

8) Hardware Assembly on the Wooden Board

1. Cut and prepare the wooden panel.
 2. Mount the NodeMCU using plastic spacers.
 3. Mount the RC522 in a way that exposes the antenna area.
 4. Mount the LCD at eye level.
 5. Mount the buzzer/LED in a visible position.
 6. Route and organize wires neatly using cable ties.
 7. Add a back cover for safety.
 8. Add a front sticker: “Place your card here”.
-

9) Testing Procedure

1. Power up the system and check Wi-Fi connection in Serial Monitor.
 2. Use the write program to store data into the card.
 3. Scan the card and verify LCD displays the student info.
 4. Check if Google Sheets log is updated.
 5. Test for duplicate scans — ensure no repeated entries.
-

10) Troubleshooting

- **LCD not displaying anything:** Check I2C address with I2C scanner.
- **RC522 not reading:** Confirm 3.3V supply, check wiring, verify authentication key.
- **ESP8266 not connecting to Wi-Fi:** Recheck SSID/password and signal range.
- **Google Script timeout:** Verify deployment URL and permissions.
- **Repeated entries:** Improve UID + timestamp filtering.

11) Future Improvements

- Add RTC or use NTP time for accurate timestamps.
 - Local caching of scans when Wi-Fi is unavailable.
 - Add authentication token for secure WebApp access.
 - Build a web dashboard instead of Google Sheets.
 - Integrate Firebase for real-time attendance monitoring.
-

12) Code Attachments

You can paste your full `.ino` and `.gs` code here if desired. Currently placeholders are left for clean formatting.

13) Security & Privacy Notes

- MIFARE 1K cards are not encrypted by default — avoid storing sensitive data.
 - Protect your Google Script endpoint from public access when possible.
-

14. Detailed Explanation of RFID Data Writing and Attendance System Codes

14.1 Overview

The system consists of two main codebases:

1. **RFID Write Program** – used to store student data inside MIFARE 1K RFID cards.
2. **RFID Attendance System Program** – used to read card data, show it on the LCD, and send it as a POST request to Google Sheets.
3. **Google Apps Script (WebApp)** – receives data from the ESP8266 and logs it in the spreadsheet.

Below is a full, technical explanation of how every part works.

14.2 RFID Memory Structure (MIFARE 1K)

MIFARE Classic 1K cards contain:

- **16 sectors**
- Each sector contains **4 blocks**
- Each block contains **16 bytes**

This gives a total of **1024 bytes of storage**. Only *data blocks* can be written. Sector Trailer blocks (every 4th block) contain keys and cannot be used for data.

In this project, the following blocks are used:

- Block **4** → Student Roll Number
- Block **5** → Student Name
- Block **6** → Department/Program
- Block **8** → Level/Group
- Block **9** → Additional info

Each block can store **16 bytes only**, so the stored strings must be short or split across multiple blocks.

14.3 RFID Write Program – Detailed Explanation

This program writes a student's information into the RFID card.

Key Functions

- `mfr522.PICC_IsNewCardPresent()` → Checks if a new card is near the reader.
- `mfr522.PICC_ReadCardSerial()` → Reads the UID of the card.
- `mfr522.PCD_Authenticate()` → Authenticates the selected block using Key A.
- `mfr522.MIFARE_Write(block, data, 16)` → Writes 16 bytes to the block.

Input Method

The user enters data over Serial Monitor. Each field ends with the character #. Example:

```
Youssef#  
30408070100553#  
G1#
```

Conversion to Bytes

Because the RFID card only accepts bytes, the string is converted:

```
inputString.getBytes(buffer, 16);
```

If the string is shorter than 16 bytes, the rest of the block is padded with zeros.

Writing Process

1. Wait for a card.
 2. Authenticate the block.
 3. Convert the string to 16-byte buffer.
 4. Write buffer to block.
 5. Display success message.
-

14.4 RFID Read + Attendance System Program – Detailed Explanation

This program runs on the ESP8266 and handles:

- Reading data from RFID
- Displaying it on LCD
- Sending attendance record to Google Sheets

RFID Reading Functions

- `mfr522.MIFARE_Read(block, buffer, &size)` → Reads 16 bytes from a block.
- `String((char*)buffer)` → Converts bytes back to readable text.

The program reads the same blocks written earlier (4, 5, 6, 8, 9).

14.5 Duplicate Scan Prevention

To avoid multiple entries for the same student if the card stays on the reader:

- The UID is stored in `lastUID`.
- The time of last scan is stored in `lastScanTime`.

The system only accepts a new scan if:

- UID is different, or
- 5 seconds have passed since last scan.

14.6 Google Sheets Communication

The ESP8266 sends data to a Google Script using **HTTPS POST**.

JSON Payload Example

```
{  
  "command": "insert_row",  
  "sheet_name": "Sheet1",  
  "values": "30408,Youssef,G1,Engineering"  
}
```

Libraries Used

- `ESP8266WiFi.h` – connects to Wi-Fi
 - `HTTPSRedirect.h` – handle
-